Міністерство освіти і науки України
Донецький національний університет імені Василя Стуса Факультет інформаційних і прикладних технологій
Кафедра інформаційних технологій

ЗВІТ

з лабораторної роботи № 16

з дисципліни «Основи програмування»

на тему:

«Побудова графіків ф-цій за допомогою віджета Canvas»

Виконав: студент гр. Б25_д/F3
Кручківський Ю.О.
Перевірив: доц. Бабаков Р. М.

Вінниця – 2025

Зміст

Визначення завдання3
Код до завдання3
Код UI/gui/TkinterViewer.py3
Діаграма5
Приклад використання модуля7
Висновки8
Джерела8

Визначення завдання

Для завдання було обрано тему “Калькулятор для виробів з тонколистових металів”. Було реалізовано три команди по заданій специфіці

- Показати доступні тонколистові метали
- Розрахувати розгортку та вартість труби
- Розрахувати приблизну вартість коліна
- Вивід графіків ф-цій за допомогою віджета Canvas

В якості основи для програми було використано раніше розроблений програмний комплекс в якості індивідуального творчого завдання

Код до завдання

Уривок коду UI/gui/TkinterViewer.py

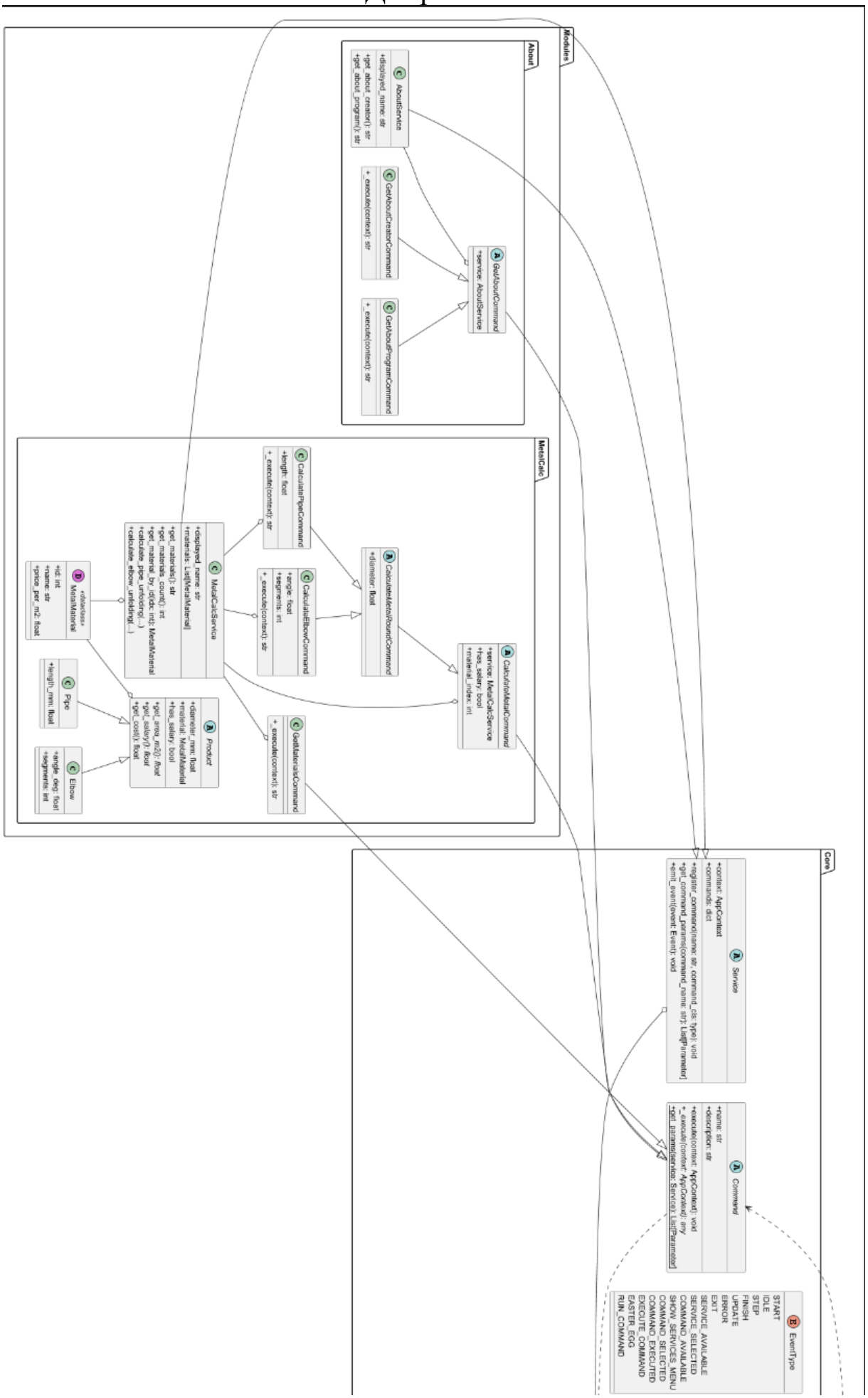
```
        def _draw_plot(self, parent, points):
            if not points:
                return

            canvas_width = 600
            canvas_height = 300
            padding = 40

            canvas = tk.Canvas(parent, width=canvas_width,
height=canvas_height, bg="white", highlightthickness=1,
highlightbackground="black")
            canvas.pack(pady=10)

            x_coords = [p[0] for p in points]
            y_coords = [p[1] for p in points]

            min_x, max_x = min(x_coords), max(x_coords)
            min_y, max_y = min(y_coords), max(y_coords)

            range_x = max_x - min_x if max_x != min_x else 1
            range_y = max_y - min_y if max_y != min_y else 1

            def scale_x(x):
                return padding + (x - min_x) * (canvas_width - 2 *
padding) / range_x

            def scale_y(y):
                return canvas_height - (padding + (y - min_y) *
(canvas_height - 2 * padding) / range_y)

            canvas.create_line(padding, canvas_height - padding,
canvas_width - padding, canvas_height - padding, arrow=tk.LAST) # X
            canvas.create_line(padding, canvas_height - padding,
padding, padding, arrow=tk.LAST) # Y

            scaled_points = [(scale_x(x), scale_y(y)) for x, y in
points]

            for i in range(len(scaled_points) - 1):
                p1 = scaled_points[i]
                p2 = scaled_points[i+1]
                canvas.create_line(p1[0], p1[1], p2[0], p2[1],
fill="blue", width=2)
```

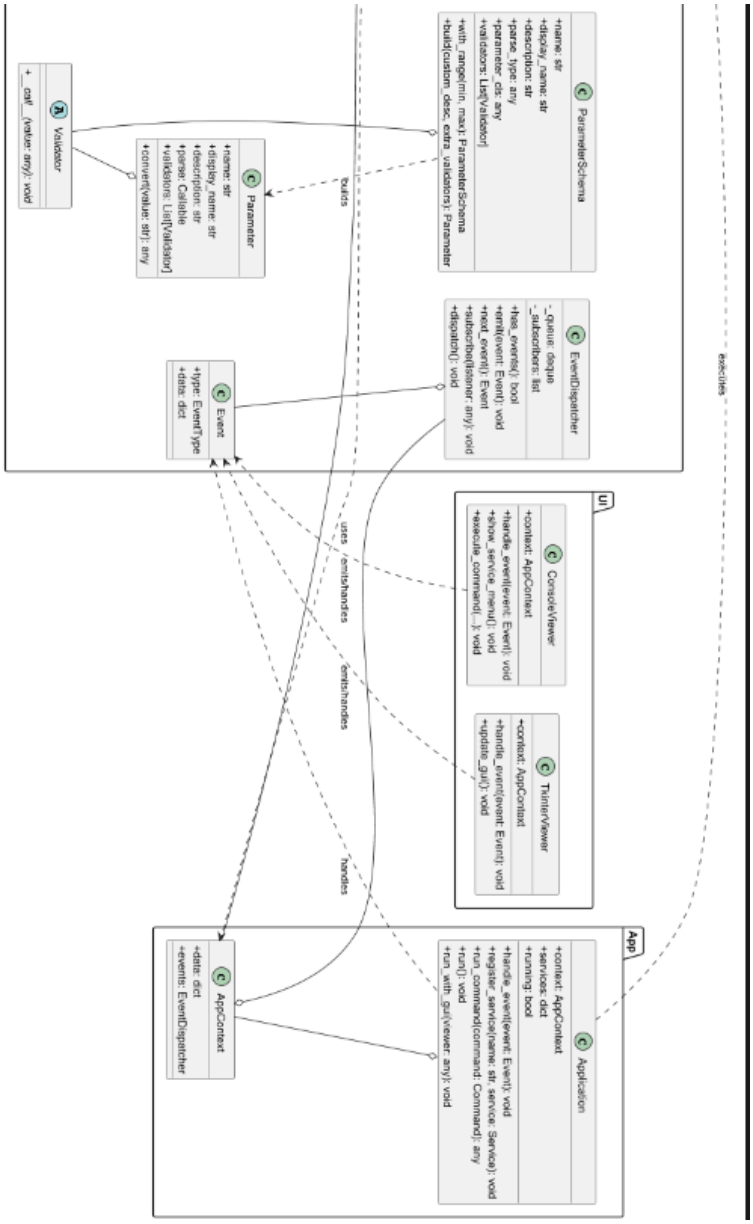
```
                canvas.create_oval(p1[0]-3, p1[1]-3, p1[0]+3, p1[1]+3,
fill="red")

            last_p = scaled_points[-1]
            canvas.create_oval(last_p[0]-3, last_p[1]-3, last_p[0]+3,
last_p[1]+3, fill="red")

            canvas.create_text(canvas_width - padding, canvas_height -
padding + 15, text="X")
            canvas.create_text(padding - 15, padding, text="Y")
```

Уривок коду /modules/metal_calc/service.py

```
        def calculate_air_flow_plot(self, diameter_mm: float,
speed_min: float = 1, speed_max: float = 10):
            print(speed_min)
            print(speed_max)
            step = 0.5
            d = diameter_mm / 1000
            area = math.pi * d ** 2 / 4

            def calc_q(v, a):
                return v * a * 3600

            points = []

            steps_count = int((speed_max - speed_min) / step)

            for i in range(steps_count + 1):
                v = speed_min + i * step
                q = calc_q(v, area)
                points.append((round(v,2), round(q,2)))

            test_points = [(x, abs(x**3) ) for x in
range(int(speed_min), int(speed_max)+1)]
            return Response(boxes=[
                PlotBox(points),
                TextBox("Точки"),
                TableBox([TableCell((0, x), str(point)) for x, point in
enumerate(points)]),
                PlotBox(test_points)])
```

Діаграма


Modules
About
AboutService
+displayed_name: str
+get_about_creator(): str
+get_about_program(): str
GetAboutCommand
+service: AboutService
GetAboutCreatorCommand
+_execute(context): str
GetAboutProgramCommand
+_execute(context): str
MetalCalc
CalculateMetalRoundCommand
+diameter: float
CalculateElbowCommand
+angle: float
+segments: int
+_execute(context): str
CalculatePipeCommand
+length: float
+_execute(context): str
CalculateMetalCommand
+service: MetalCalcService
+has_salary: bool
+material_index: int
GetMaterialsCommand
+_execute(context): str
MetalCalcService
+displayed_name: str
+materials: List[MetalMaterial]
+get_materials(): str
+get_materials_count(): int
+get_material_by_id(idc int): MetalMaterial
+calculate_pipe_unfolding(...)
+calculate_elbow_unfolding(...)
Product
+diameter_mm: float
+material: MetalMaterial
+has_salary: bool
+get_area_m2(): float
+get_salary(): float
+get_cost(): float
«dataclass»
MetalMaterial
+id: int
+name: str
+price_per_m2: float
Pipe
+length_mm: float
Elbow
+angle_deg: float
+segments: int
Core
Service
+context: AppContext
+commands: dict
+register_command(name: str, command_cls: type): void
+get_command_param(command_name: str): List[Parameter]
+emit_event(event: Event): void
Command
+name: str
+description: str
+execute(context: AppContext): void
+_execute(context: AppContext): any
+get_parameter(service: Service): List[Parameter]
EventType
START
IDLE
STEP
FINISH
UPDATE
ERROR
EXIT
SERVICE_AVAILABLE
SERVICE_SELECTED
COMMAND_AVAILABLE
SHOW_SERVICES_MENU
COMMAND_SELECTED
COMMAND_EXECUTED
EXECUTE_COMMAND
EASTER_EGG
RUN_COMMAND


ParameterSchema
+name: str
+display_name: str
+description: str
+parse_type: any
+parameter_cls: any
+validators: List[Validator]
+with_range(min, max): ParameterSchema
+build(custom_desc, extra_validators): Parameter
EventDispatcher
_queue: deque
_subscribers: list
+has_events(): bool
+emit(event: Event): void
+next_event(): Event
+subscribe(listener: any): void
+dispatch(): void
builds
Parameter
+name: str
+display_name: str
+description: str
+parser: Callable
+validators: List[Validator]
+convert(value: str): any
Event
+type: EventType
+data: dict
Validator
+__call__(value: any): void
uses
emits/handles
emits/handles
handles
executes
UI
ConsoleViewer
+context: AppContext
+handle_event(event: Event): void
+show_service_menu(): void
+execute_command(...): void
TkinterViewer
+context: AppContext
+handle_event(event: Event): void
+update_gui(): void
App
Application
+context: AppContext
+services: dict
+running: bool
+handle_event(event: Event): void
+register_service(name: str, service: Service): void
+run_command(command: Command): any
+run(): void
+run_with_gui(viewer: any): void
AppContext
+data: dict
+events: EventDispatcher

Приклад використання модуля

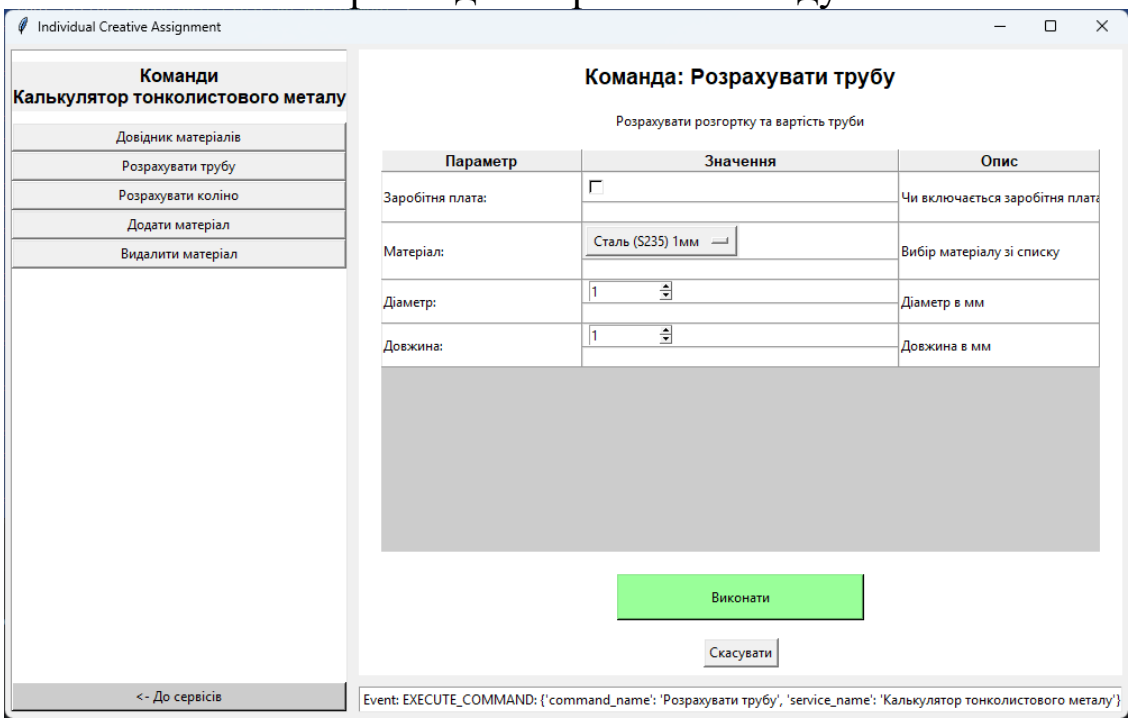


Рисунок 1 - приклад форми команди

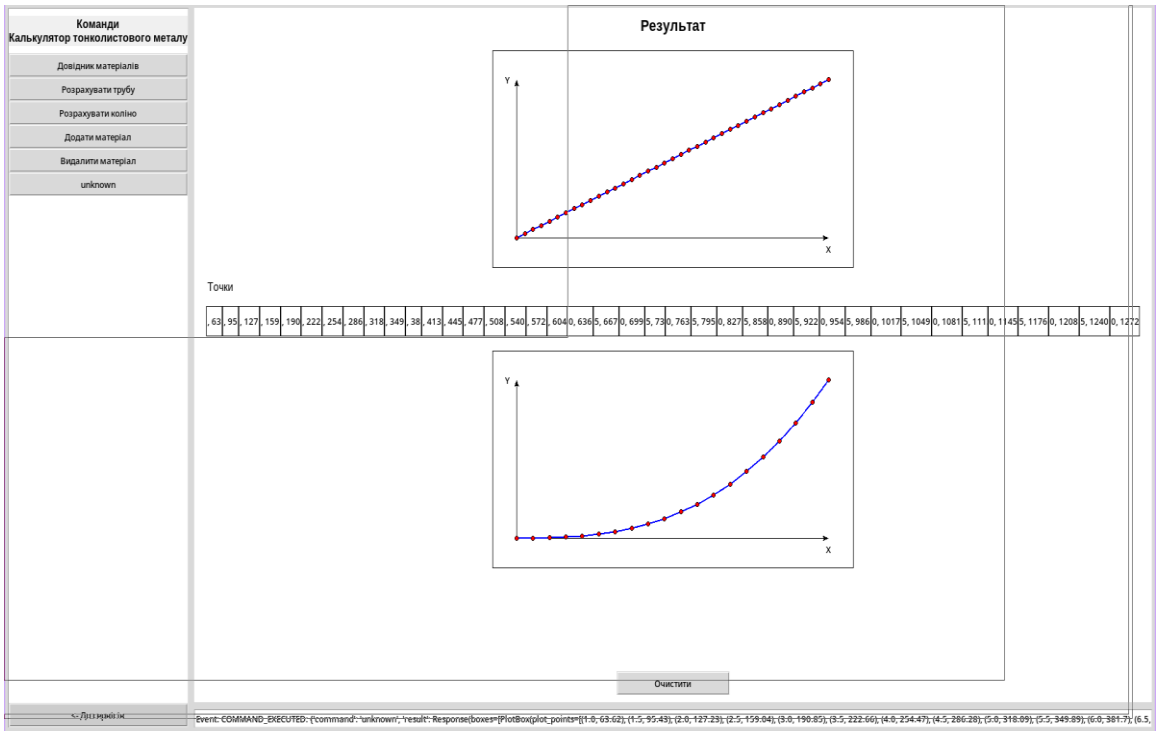


Рисунок 2 - приклад коректного виводу

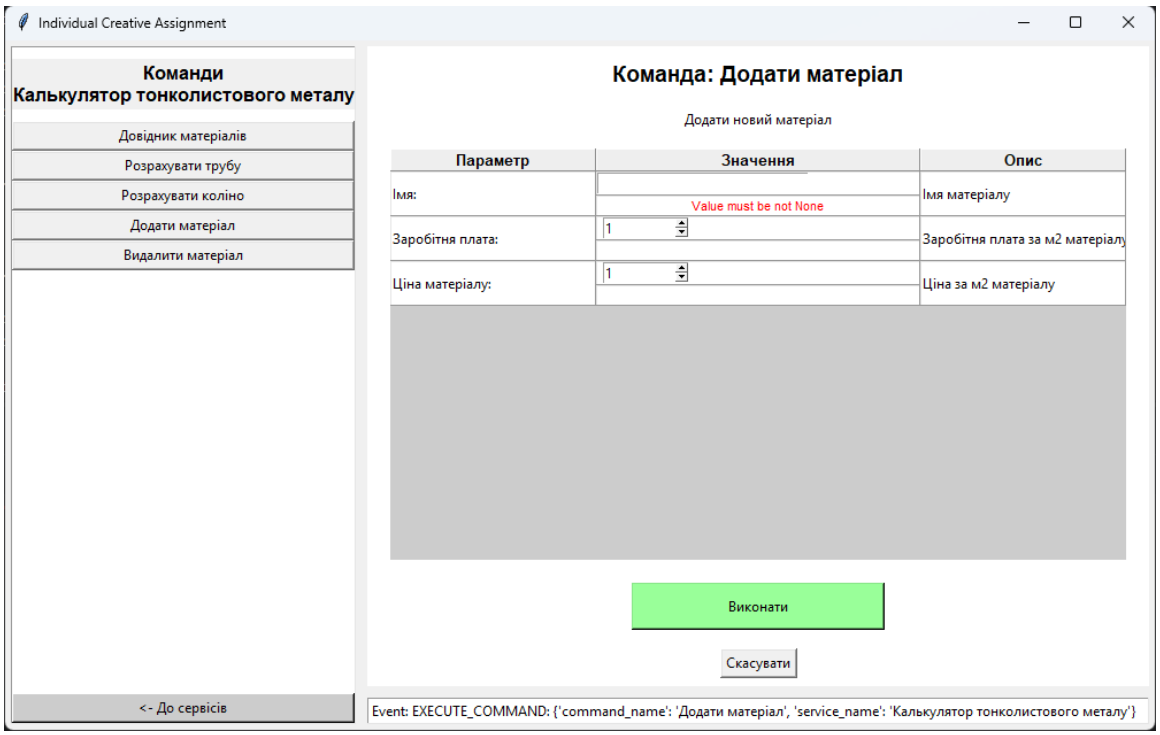


Рисунок 3 - приклад помилки

Висновки

В ході виконання лабораторної було допрацьовано програмний комплекс та розроблено модуль для лабораторної роботи

Джерела

- Посилання на репозиторій https://github.com/Figeiron/IndividualCreativeAssignment/releases/tag/LabWork_14-v0.0.2